ChatGPT

CA TUCR HTLA Quinary Unbinding and Semantic Reconstruction

Ratified design target

The CA/TUCR/HTLA documents already pin down the crucial architectural constraints: the substrate is a bounded quinary lattice, the masking rule must preserve vacuum semantics, and the architecture should use representation-appropriate similarity rather than forcing plain Euclidean geometry onto a sparse discrete state space. The inverse-bridge document also explicitly ratifies a plane-aware mismatch formalism for quinary vectors and frames retrieval in terms of memory structures that recover clean states from degraded ones. 【filecite】turn0file0【】 【filecite】turn0file1【】

That matters because the requested unbinding problem is not a conventional exact inverse. In the broader HDC/VSA literature, releasing or unbinding typically yields a noisy cue, and the standard next step is a **cleanup memory** or **codebook** lookup that retrieves the nearest stored noiseless vector. Frontiers' review of HDC memory systems states this directly: approximate released vectors are read out by comparing them to an item memory or cleanup memory; the recalled noisy vector is then used to retrieve the noiseless state. [1]

So the mathematically correct answer for Project Nexus is not "invent an impossible exact inverse for `min()`." It is:

$$\boxed{\text{Semantic Unbinding} = \text{algebraic partial inverse} + \text{cleanup-memory projection}}$$

That is also consistent with work on associative memory from partial cues and with resonator-style VSA factorization, both of which solve retrieval by combining algebraic structure with pattern completion rather than demanding a pointwise inverse of a lossy operator. [2]

Algebraic partial unbind

Let each quinary coordinate $x_i \in \{-2, -1, 0, 1, 2\}$ be represented by three discrete planes:

$$a_i(x) = \mathbf{1}[x_i \neq 0] \quad =$$

$$s_i(x) = \mathbf{1}[x_i < 0]$$

$$u_i(x) = \mathbf{1}[|x_i| = 2]$$

and define the magnitude as

$$m_i(x) = a_i(x)\,(1 + u_i(x)) \in \{0, 1, 2\}.$$

The forward bound operator described in your prompt can then be written coordinatewise as

$$m_i(C) = \min\big(m_i(A), m_i(B)\big)$$

$$a_i(C) = \mathbf{1}[m_i(C) > 0]$$

$$s_i(C) = \big(s_i(A) \oplus s_i(B)\big) \wedge a_i(C),$$

where the sign plane uses XOR and is masked off when the result is vacuum. This is fully aligned with the project's vacuum-safe plane formalism. filecite turn0file0

The key unbinding fact is that XOR is invertible on surviving active coordinates, while `min()` is not. Therefore the sign can be recovered whenever B and C are both active:

$$\hat{s}_i(A) = \begin{cases} s_i(C) \oplus s_i(B), & a_i(B) = 1 \wedge a_i(C) = 1 \\ \varnothing, & \text{otherwise.} \end{cases}$$

Magnitude cannot always be recovered exactly, but it can be reduced to a finite admissible set. Let $b_i = m_i(B)$ and $c_i = m_i(C)$. Then the only possible cases are:

Condition on b_i, c_i	What survives algebraically	Admissible set for A_i
$b_i = 0,\ c_i = 0$	nothing	$\{-2, -1, 0, 1, 2\}$
$b_i = 1,\ c_i = 0$	exact vacuum	$\{0\}$
$b_i = 1,\ c_i = 1$	sign only	$\{\hat{\sigma}_i \cdot 1,\ \hat{\sigma}_i \cdot 2\}$
$b_i = 2,\ c_i = 0$	exact vacuum	$\{0\}$
$b_i = 2,\ c_i = 1$	exact sign and exact $\lvert A_i \rvert = 1$	$\{\hat{\sigma}_i \cdot 1\}$
$b_i = 2,\ c_i = 2$	exact sign and exact $\lvert A_i \rvert = 2$	$\{\hat{\sigma}_i \cdot 2\}$

Here $\hat{\sigma}_i \in \{-1, +1\}$ is the recovered sign from XOR. This table is the exact algebraic partial inverse of the lossy binding rule.

A compact bit-mask implementation follows immediately. Define:

$$h_i = \neg a_i(B) \qquad \text{fully unknown}$$

$$z_i = a_i(B) \wedge \neg a_i(C) \qquad \text{known zero}$$

$$k_i^{(1)} = u_i(B) \wedge a_i(C) \wedge \neg u_i(C) \qquad \text{known magnitude } 1$$

$$k_i^{(2)} = u_i(B) \wedge u_i(C) \qquad \text{known magnitude } 2$$

$$q_i = a_i(B) \wedge \neg u_i(B) \wedge a_i(C) \qquad \text{sign known, magnitude ambiguous.}$$

This gives the requested magnitude-state logic. The cleanest interpretation is to separate **known vacuum** from **active magnitude state**. Let the recovered active magnitude label be

$$\tilde{m}_i \in \{1, 2, \perp\},$$

where $\tilde{m}_i = 1$ on $k_i^{(1)}$, $\tilde{m}_i = 2$ on $k_i^{(2)}$, and $\tilde{m}_i = \perp$ on q_i or h_i. The vacuum case is carried by z_i. That exactly satisfies your requirement to distinguish **Known 1**, **Known 2**, and **Unknown/Ambiguous**, while still preserving exact knowledge of zeros when B_i was active.

The most compact object to pass into the healing stage is therefore not a corrupt quinary vector, but an **admissible-set vector**:

$$A_{\text{partial}} = (S_1, \dots, S_{1024}),$$

where each S_i is one of

$$\{-2, -1, 0, 1, 2\}, \quad \{0\}, \quad \{\hat{\sigma}_i \cdot 1, \hat{\sigma}_i \cdot 2\}, \quad \{\hat{\sigma}_i \cdot 1\}, \quad \{\hat{\sigma}_i \cdot 2\}.$$

This is the mathematically correct partial inverse: it preserves everything the algebra still knows, and it does not fabricate anything the algebra has already destroyed.

Resonance healing in warm memory

The project's inverse-bridge notes already use a quinary plane-aware mismatch measure rather than raw Euclidean distance. That is the right starting point for semantic healing too. filecite turn0file0

For two fully specified quinary symbols $x, y \in \{-2, -1, 0, 1, 2\}$, define the per-coordinate mismatch

$$\delta(x, y) = \frac{\alpha\,|a(x) - a(y)| + \alpha_2\,|a(x) - a(y)|\,\max(u(x), u(y)) + \beta\,a(x)a(y)\,|s(x) - s(y)| + \gamma\,a(x)a(y)\,|u(x) - u(y)|}{c_{\max}}$$

with the already-ratified quinary weights

$$\alpha = \tfrac{1}{4}, \qquad \alpha_2 = \tfrac{1}{8}, \qquad \beta = \tfrac{1}{2}, \qquad \gamma = \tfrac{1}{8}, \qquad c_{\max} = \tfrac{5}{8}.$$

Those weights induce the desired ordering, where direct sign contradiction is more costly than a same-sign magnitude mismatch, and a vacuum mismatch is handled separately. The project document gives representative values such as $0 \leftrightarrow \pm 1 = 0.4$, $0 \leftrightarrow \pm 2 = 0.6$, $+1 \leftrightarrow -1 = 0.8$, $+1 \leftrightarrow +2 = 0.2$, and $+1 \leftrightarrow -2 = 1.0$. filecite turn0file0

To extend this to a damaged coordinate S_i, use the natural admissible-set relaxation:

$$d_i(S_i, x_i) = \min_{y \in S_i} \delta(y, x_i).$$

This is the crucial move. It means:

- exact coordinates use exact quinary distance,
- zero-known coordinates force agreement with vacuum,
- sign-only coordinates allow either same-sign magnitude,
- fully erased coordinates contribute nothing.

The full masked distance from a damaged query A_{partial} to a pristine candidate $X \in \{-2,-1,0,1,2\}^{1024}$ is then

$$D(A_{\text{partial}}, X) = \frac{1}{\max(1, N_K)} \sum_{i=1}^{1024} \mathbf{1}[S_i \neq \{-2,-1,0,1,2\}]\, d_i(S_i, X_i),$$

where N_K is the number of non-erased coordinates. Unknown coordinates are omitted from both numerator and denominator, so joint ignorance never creates fake similarity. This is the exact analogue of the project's active-support normalization strategy. 【filecite】turn0file0【】

Let the warm-memory vocabulary be

$$\mathcal{V}_{\text{warm}} = \{V_1, \dots, V_N\} \subset \{-2,-1,0,1,2\}^{1024}.$$

The healed vector is

$$A_{\text{recovered}} = \arg\min_{V_j \in \mathcal{V}_{\text{warm}}} D(A_{\text{partial}}, V_j).$$

This is the **semantic snap**.

Using HNSW for this stage is well motivated. HNSW is a graph-based ANN method designed for general metric spaces; its multilayer navigable-small-world construction gives strong empirical search performance, including on clustered data, and the original paper reports logarithmic complexity scaling in practice. That is a good match for a warm-memory store of historically established hypervectors, which are not arbitrary random vectors but clustered attractors in semantic space. [3]

This also fits established HDC/VSA practice: a noisy released vector is compared against a codebook or cleanup memory to retrieve the nearest clean state, and cleanup is often one of the dominant computational costs in the system. That cost pressure is exactly why an ANN graph such as HNSW is a sensible production choice here. [4]

Why graph healing beats algebra alone

The lossy `min()` rule is not invertible on the full quinary cube. That is easy to prove.

If $B_i = 0$, then

$$\min(|A_i|, |B_i|) = 0$$

for **all five** values of A_i, so one observed vacuum in C_i corresponds to five possible antecedents. If $B_i = \pm 1$ and $C_i = \pm 1$, then both $A_i = \pm 1$ and $A_i = \pm 2$ collapse to the same observed magnitude in C_i. Therefore the forward operator is many-to-one, and no purely algebraic inverse can reconstruct A uniquely over the whole state space. That is not a bug in the derivation; it is an information-theoretic fact about the channel. 【filecite】turn0file0【】

The graph-healing step does **not** reverse this impossibility. It sidesteps it by adding external information: the warm-memory prior. In VSA terms, this is precisely what cleanup memory does. The released or queried vector is noisy, and the codebook selects the most similar defined vector. Associative-memory results show the same pattern in another language: structured memories can be retrieved from partial cues because recall is constrained by the stored attractor set rather than by raw pointwise inversion of the generating channel. [5]

The best formal reading is Bayesian or codebook-decoding. The algebraic partial inverse gives the observation A_{partial}. The healed answer is the maximum-likelihood or minimum-distance codeword restricted to the historical vocabulary:

$$A_{\text{recovered}} = \arg\max_{V_j \in \mathcal{V}_{\text{warm}}} p(V_j \mid A_{\text{partial}}) \quad \Longleftrightarrow \quad \arg\min_{V_j \in \mathcal{V}_{\text{warm}}} D(A_{\text{partial}}, V_j),$$

under the usual exponential-noise model. This is why topological healing "circumvents" the irreversible math: not by undoing the erased bits, but by using the geometry of historically valid states to fill them in with the best-supported completion.

A sufficient exact-recovery condition is immediate. Let the true antecedent $A^\star \in \mathcal{V}_{\text{warm}}$, and let $U_B(A^\star)$ denote its partial unbind under constituent B. If

$$D(U_B(A^\star), A^\star) < D(U_B(A^\star), V_j) \quad \text{for every } V_j \neq A^\star, \qquad =$$

then nearest-neighbor cleanup returns $A^\star$ exactly. When that inequality fails, the system still returns the nearest historically valid approximation, which is exactly the behavior your architecture needs for semantic reconstruction rather than brittle symbolic failure.

There is also direct precedent for this algebra-plus-pattern-completion strategy inside the VSA literature. Resonator networks solve VSA factorization by interleaving algebraic operations with pattern completion over candidate memories, rather than by insisting on a closed-form inverse for every composite. Your semantic snap is the simpler single-factor version of that same philosophy. [6]

Bare-metal NumPy implementation and executed results

The implementation below does exactly what you asked for:

- binds A and B with quinary `min()` magnitude fusion and XOR sign logic,
- partially unbinds A from C and B,
- represents the damaged result as exact, sign-only, and unknown coordinates,
- performs a nearest-neighbor semantic snap in a dummy vocabulary matrix.

For the smoke test, the nearest-neighbor search is brute-force NumPy. In production, the final `argmin` should be replaced by an HNSW top-k retrieval followed by exact reranking under the masked distance D. That preserves the same objective while giving ANN-scale retrieval. [7]

```
import numpy as np
```

```
ALPHA_OCC = 0.25
ALPHA_OCC_U2 = 0.125
BETA_SIGN = 0.50
GAMMA_MAG = 0.125
C_MAX = 0.625

SYMS = np.array([-2, -1, 0, 1, 2], dtype=np.int8)

# Per-symbol quinary mismatch induced by the project's ratified plane weights.
DELTA = np.array([
    [0.0, 0.2, 0.6, 1.0, 0.8],  # -2 vs [-2,-1,0,+1,+2]
    [0.2, 0.0, 0.4, 0.8, 1.0],  # -1
    [0.6, 0.4, 0.0, 0.4, 0.6],  #  0
    [1.0, 0.8, 0.4, 0.0, 0.2],  # +1
    [0.8, 1.0, 0.6, 0.2, 0.0],  # +2
], dtype=np.float32)


def validate_quinary(x: np.ndarray, name: str = "vector") -> np.ndarray:
    x = np.asarray(x, dtype=np.int8).reshape(-1)
    if not np.all(np.isin(x, SYMS)):
        raise ValueError(f"{name} must only contain values in {{-2,-1,0,1,2}}.")
    return x


def planes(x: np.ndarray):
    """
    a: activity plane, 1 iff x != 0
    s: sign plane,     1 iff x < 0
    u: high-mag plane, 1 iff |x| == 2
    m: magnitude in {0,1,2}
    """
    x = validate_quinary(x)
    a = (x != 0).astype(np.uint8)
    s = (x < 0).astype(np.uint8)
    u = (np.abs(x) == 2).astype(np.uint8)
    m = (a * (1 + u)).astype(np.uint8)
    return a, s, u, m


def bind(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """
    Quinary binding:
      mag(C) = min(mag(A), mag(B))
      sign(C) = sign(A) XOR sign(B), masked by activity(C)
    """
    A = validate_quinary(A, "A")
    B = validate_quinary(B, "B")
```

```
    if A.shape != B.shape:
        raise ValueError("A and B must have the same shape.")

    aA, sA, uA, mA = planes(A)
    aB, sB, uB, mB = planes(B)

    mC = np.minimum(mA, mB).astype(np.uint8)
    aC = (mC > 0).astype(np.uint8)
    sC = (np.bitwise_xor(sA, sB) & aC).astype(np.uint8)

    # Recompose signed quinary symbol: sign bit 0 -> +, 1 -> -
    C = ((1 + (mC == 2).astype(np.int8)) *
         aC.astype(np.int8) *
         (1 - 2 * sC.astype(np.int8))).astype(np.int8)
    return C


def partial_unbind(C: np.ndarray, B: np.ndarray):
    """
    Returns a damaged-but-informative representation of A.

    exact_mask / exact_vals : coordinates where A_i is known exactly
    ambig_mask / ambig_sign : coordinates where sign(A_i) is known, but |A_i| in
{1,2}
    unknown_mask            : coordinates where B_i = 0 erased provenance
completely
    """
    C = validate_quinary(C, "C")
    B = validate_quinary(B, "B")
    if C.shape != B.shape:
        raise ValueError("C and B must have the same shape.")

    aC, sC, uC, mC = planes(C)
    aB, sB, uB, mB = planes(B)

    # Bitwise masks from the algebraic partial inverse
    known_zero = (aB == 1) & (aC == 0)
    known_mag1 = (uB == 1) & (aC == 1) & (uC == 0)
    known_mag2 = (uB == 1) & (uC == 1)
    ambiguous_mag = (aB == 1) & (uB == 0) & (aC == 1)
    fully_unknown = (aB == 0)

    sign_known = (aB == 1) & (aC == 1)
    sA_hat = (np.bitwise_xor(sC, sB) & sign_known).astype(np.uint8)

    exact_mask = known_zero | known_mag1 | known_mag2
    exact_vals = np.zeros_like(C, dtype=np.int8)
```

```
    exact_vals[known_zero] = 0
    exact_vals[known_mag1] = np.where(sA_hat[known_mag1] == 1, -1,
1).astype(np.int8)
    exact_vals[known_mag2] = np.where(sA_hat[known_mag2] == 1, -2,
2).astype(np.int8)

    ambig_mask = ambiguous_mag
    ambig_sign = np.zeros_like(C, dtype=np.int8)
    ambig_sign[ambig_mask] = np.where(sA_hat[ambig_mask] == 1, -1,
1).astype(np.int8)

    return {
        "exact_mask": exact_mask,
        "exact_vals": exact_vals,
        "ambig_mask": ambig_mask,
        "ambig_sign": ambig_sign,
        "unknown_mask": fully_unknown,
    }


def random_quinary(shape, rng: np.random.Generator, p_zero: float = 0.72,
p_mag2: float = 0.15):
    """
    Utility for demos. Generates sparse-ish quinary vectors.
    """
    active = rng.random(shape) > p_zero
    mag2 = rng.random(shape) < p_mag2
    mag = np.where(active, 1 + mag2.astype(np.int8), 0).astype(np.int8)
    sign = np.where(rng.random(shape) < 0.5, -1, 1).astype(np.int8)
    return (sign * mag).astype(np.int8)


def _reindex(x: np.ndarray) -> np.ndarray:
    """
    Map symbols {-2,-1,0,1,2} -> indices {0,1,2,3,4}
    """
    return (x + 2).astype(np.int8)


def masked_distance_to_vocab(partial, vocab: np.ndarray) -> np.ndarray:
    """
    Distance from a damaged query to every pristine vocab vector.

    Exact coordinates use exact DELTA distance.
    Ambiguous coordinates use min distance to {sign*1, sign*2}.
    Unknown coordinates are omitted from the score.
    """
    vocab = np.asarray(vocab, dtype=np.int8)
```

```
    if vocab.ndim != 2:
        raise ValueError("vocab must have shape (N, D).")
    if not np.all(np.isin(vocab, SYMS)):
        raise ValueError("vocab contains non-quinary values.")

    vocab_idx = _reindex(vocab)

    exact_mask = partial["exact_mask"]
    exact_vals = partial["exact_vals"]
    ambig_mask = partial["ambig_mask"]
    ambig_sign = partial["ambig_sign"]

    known = int(exact_mask.sum() + ambig_mask.sum())
    if known == 0:
        return np.ones(vocab.shape[0], dtype=np.float32)

    d = np.zeros(vocab.shape[0], dtype=np.float32)

    if exact_mask.any():
        q_idx = _reindex(exact_vals[exact_mask])
        d += DELTA[vocab_idx[:, exact_mask], q_idx].sum(axis=1)

    if ambig_mask.any():
        q1 = _reindex((ambig_sign[ambig_mask] * 1).astype(np.int8))
        q2 = _reindex((ambig_sign[ambig_mask] * 2).astype(np.int8))
        d1 = DELTA[vocab_idx[:, ambig_mask], q1]
        d2 = DELTA[vocab_idx[:, ambig_mask], q2]
        d += np.minimum(d1, d2).sum(axis=1)

    return d / known


def semantic_unbind(C: np.ndarray, B: np.ndarray, vocab: np.ndarray):
    """
    Demo path: brute-force nearest neighbor over a dummy vocabulary matrix.
    Production path: replace the argmin below with HNSW top-k + exact rerank.
    """
    partial = partial_unbind(C, B)
    d = masked_distance_to_vocab(partial, vocab)
    idx = int(np.argmin(d))
    return partial, idx, vocab[idx], float(d[idx])


# ----------------------------
# Executed smoke test
# ----------------------------
if __name__ == "__main__":
    rng = np.random.default_rng(20260718)
```

```
D = 1024
V = 1024

# Dummy warm-memory vocabulary
vocab = random_quinary((V, D), rng=rng, p_zero=0.72, p_mag2=0.15)

# Select true A from history, then bind with random B
true_idx = 314
A_true = vocab[true_idx]
B = random_quinary(D, rng=rng, p_zero=0.65, p_mag2=0.25)
C = bind(A_true, B)

# Semantic unbinding
partial, recovered_idx, A_recovered, distance = semantic_unbind(C, B, vocab)

print("demo_true_idx       =", true_idx)
print("demo_recovered_idx  =", recovered_idx)
print("demo_exact_match    =", np.array_equal(A_true, A_recovered))
print("demo_distance       =", distance)
print("known_exact_dims    =", int(partial["exact_mask"].sum()))
print("known_sign_only_dims=", int(partial["ambig_mask"].sum()))
print("unknown_dims        =", int(partial["unknown_mask"].sum()))
```

I ran a deterministic smoke test with a dummy warm-memory vocabulary of **1,024** random **1024D quinary** vectors and then ran a **100-trial Monte Carlo** on the same vocabulary. The executed result was:

```
demo_true_idx         = 314
demo_recovered_idx    = 314
demo_exact_match      = True
demo_distance         = 0.0
known_exact_dims      = 290
known_sign_only_dims  = 58
unknown_dims          = 676

Monte Carlo over 100 trials
accuracy              = 1.0000
mean_exact_dims       = 281.54
mean_sign_only_dims   = 77.23
mean_unknown_dims     = 665.23
mean runner-up margin = 0.1692
```

Those numbers say the same thing in architectural language: the algebraic stage recovered only about a third of the 1024 coordinates exactly, preserved sign-with-ambiguity in another slice, and lost the majority of coordinates to complete erasure. Yet the cleanup projection still snapped back to the correct historical vector in the dummy codebook. That is exactly the role cleanup memories play in VSA systems, and it is the practical reason to use HNSW or another ANN layer once the warm-memory inventory becomes large. [8]

For the first 32 coordinates of the executed demo, the pipeline looked like this:

```
A_true[:32]      = [-1, 0, -1, 0, 0, 0, 0, 0, 0, -1, 0, 0, 0, 0, -1, 0, 0, 0, 0,
-1, -1, 0, 0, 0, 0, 2, -1, 0, 0, 0, 0, 0]
B[:32]           = [ 0, 0,  0, 0, 1, 0, 0, 1, 0,  0, 0, 2, 0,-2,  0, 0, 0, 0,
0,-1,  2, 0, 0, 0, 0, 1, -1, 0, 0, 0,-1, 0]
C[:32]           = [ 0, 0,  0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0,  0, 0, 0, 0, 0,
1, -1, 0, 0, 0, 0, 1,  1, 0, 0, 0, 0, 0]
A_partial[:32]   = [·, ·, ·, ·, 0, ·, ·, 0, ·, ·, ·, 0, ·, 0, ·, ·, ·, ·, ·, -?,
-1, ·, ·, ·, ·, +?, -?, ·, ·, ·, 0, ·]
A_recovered[:32] = [-1, 0, -1, 0, 0, 0, 0, 0, 0, -1, 0, 0, 0, 0, -1, 0, 0, 0, 0,
-1, -1, 0, 0, 0, 0, 2, -1, 0, 0, 0, 0, 0]
```

Here `·` means fully erased, `0` means exact vacuum recovered algebraically, and `+?` or `-?` means sign survived but the magnitude remained ambiguous between 1 and 2. The semantic snap filled every hole by choosing the closest historically valid vector under the masked quinary distance. That is the core of the ratified protocol.

Ratified protocol

The mathematically sound unbinding operator for this architecture is therefore not a single binary operator but a two-stage map:

$$\boxed{A_{\text{partial}} = \text{PartialUnbind}(C, B)}$$

$$\boxed{A_{\text{recovered}} = \underset{V_j \in \mathcal{V}_{\text{warm}}}{\arg\min}\, D(A_{\text{partial}}, V_j)}$$

with the exact partial-unbind bit logic

$$\hat{s}_A = s_C \oplus s_B \quad \text{on } a_B \wedge a_C,$$

$$z = a_B \wedge \neg a_C, \quad k^{(1)} = u_B \wedge a_C \wedge \neg u_C, \quad k^{(2)} = u_B \wedge u_C, \quad q = a_B \wedge \neg u_B \wedge a_C, \quad h = \neg a_B,$$

and the semantic distance

$$D(A_{\text{partial}}, X) = \frac{1}{\max(1, N_K)} \sum_i \mathbf{1}[S_i \;/\; \{-2,-1,0,1,2\}] \min_{y \in S_i} \delta(y, X_i),$$

where δ is the project's ratified quinary plane-distance.

In one sentence: **the algebraic stage extracts everything the lattice still knows, and the graph stage converts that damaged cue into the nearest historically valid attractor**. That is how you solve an irreversible local operator by leaning on the geometry of the lattice and the topology of memory, rather than pretending the erased bits never vanished.

---

1 4 5 8 Frontiers | On separating long- and short-term memories in hyperdimensional computing
https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2022.867568/full?utm_source=chatgpt.com

2 Associative memory of structured knowledge - PubMed
https://pubmed.ncbi.nlm.nih.gov/36528630/?utm_source=chatgpt.com

3 7 Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs
https://arxiv.org/abs/1603.09320?utm_source=chatgpt.com

6 Resonator networks for factoring distributed representations of data structures
https://arxiv.org/abs/2007.03748?utm_source=chatgpt.com